Reviewer Pack — Minimal Symmetric Tensor Norm and AC0 Parity Circuit Size

Entry 939c9435791b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 00:43:15 UTC

Minimal Symmetric Tensor Norm and AC0 Parity Circuit Size

Entry ID: `939c9435791b` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-22 00:43:15 UTC

1. Conjecture

Field A (mathematical branch): Symmetric Function Theory **Field B** (complexity object): AC^0 Circuit Complexity

Statement:

> For any AC^0 circuit C computing PARITY on n inputs, the minimal symmetric tensor norm of the tensor representation of C's output function is at least Ω(n^2 log n).

Rationale (proposer's reasoning):

> Symmetric function theory provides a rich framework for studying polynomial invariants, which may offer new insights into the structure of AC^0 circuits and their ability to compute PARITY. A symmetric tensor norm could serve as a natural invariant with respect to permutation actions on the circuit outputs.

Taxonomy category: `AC0_PARITY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `ca8d97a0d5cf1b94`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all generated AC^0 circuits C computing PARITY on n inputs (n ≤ 40), the minimal symmetric tensor norm of T_f, the tensor representation of f, is at least Ω(n^2 log n) with a 95% confidence level.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "symmetric tensor norm" AND "AC0 parity circuit size" - "PARITY circuit complexity" AND "symmetric function theory" - "tensor representation" AND AC0 AND symmetric function

Top relevant hits considered: - [s2:a647578e190c85f04e4b4751ff7434458f23add9] UNiTE: Unitary N-body Tensor Equivariant Network with Applications to Quantum Chemistry - [s2:10.31219/osf.io/qu8rz] Tensor volume exploration using attribute space representatives

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.3s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_ac0_circuit(n):
        # Simplified AC0 circuit for PARITY on n inputs
        return [random.choice([0, 1]) for _ in range(2**n)]

    def compute_tensor_representation(circuit):
        # Represent the circuit output as a tensor over the symmetric algebra of boolean functions
        n = int(math.log2(len(circuit)))
        T_f = [[0] * (2**n) for _ in range(2**n)]
        for i in range(2**n):
            for j in range(2**n):
                T_f[i][j] = circuit[(i & j) ^ ((i | j) >> 1)]
        return T_f

    def calculate_symmetric_tensor_norm(T_f, n):
        # Calculate the minimal symmetric tensor norm of T_f
        norm = 0
        for i in range(2**n):
            for j in range(2**n):
                norm += abs(T_f[i][j])
        return norm / (2**(2*n))

    def is_ac0_circuit(circuit, n):
        # Check if the circuit computes PARITY on n inputs
        for i in range(1 << n):
            input_bits = [i >> j & 1 for j in range(n)]
            output = circuit[i]
            expected_output = sum(input_bits) % 2
```

```
            if output != expected_output:
                return False
        return True

    n_values = [5, 10, 15, 20, 30, 40]
    total_norm = 0
    instances_tested = 0

    for n in n_values:
        circuit = generate_ac0_circuit(n)
        if not is_ac0_circuit(circuit, n):
            return {
                "metric_name": "symmetric_tensor_norm",
                "metric_value": None,
                "instances_tested": 0,
                "conjecture_holds": False,
                "counterexample": "not_ac0_circuit"
            }
        T_f = compute_tensor_representation(circuit)
        norm = calculate_symmetric_tensor_norm(T_f, n)
        total_norm += norm
        instances_tested += 1

    mean_norm = total_norm / len(n_values)
    conjecture_holds = mean_norm >= n**2 * math.log(n) for n in n_values
    counterexample = "" if conjecture_holds else "mean_norm < n^2 log n"

    return {
        "metric_name": "symmetric_tensor_norm",
        "metric_value": mean_norm,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else list(range(2, 50, 2))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_norm = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(res
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_norm} std=0.0 support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_norm} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mean_norm < n^2 log n\" first_failing_seed={fir
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test timed out before producing data, which means we cannot confirm or refute the conjecture with the pre-registered support condition. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13312
2	preregistration	ollama_remote	glm4:latest	0	0	10062
3	novelty	ollama_remote	glm4:latest	0	0	8138
4	novelty	ollama_remote	glm4:latest	0	0	9405
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14176
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8989
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8845
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10301
9	judge	ollama_remote	glm4:latest	0	0	26897

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 110126 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/939c9435791b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/939c9435791b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/939c9435791b.tar.gz` (if generated)